

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Título del Proyecto:

Invernadero Inteligente IoT

PONDERACIÓN: 40 pts



Tiempo estimado: 60 hrs

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Habilidad(es) blandas a formar.....	3
2. Resultado del Aprendizaje.....	3
2.1. Objetivo SMART.....	3
3. Resumen Ejecutivo.....	4
4. Enunciado del Proyecto.....	4
4.1 Descripción del problema o necesidad a resolver.....	4
4.2 Alcance del proyecto.....	4
4.3 Recursos y herramientas a utilizar.....	4
4.4 Entregables.....	4
5. Material de apoyo.....	5
6. Metodología.....	5
7. Cronograma.....	6
8. Rúbrica de Calificación.....	6
8.1 Requisitos para optar a la calificación.....	6
8.2 Detalle de la Calificación.....	7
8.3 Comentarios Generales.....	8

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Responsabilidad y trabajo colaborativo	Se aplica mediante la planificación, distribución de tareas, integración de sensores, actuadores, comunicación IoT, dashboard, base de datos y documentación técnica. Cada integrante deberá aportar a un subsistema específico y comprender el funcionamiento general del invernadero.

1.2. Competencia(s)

Con la elaboración de este proyecto, el estudiante adquirirá la siguiente competencia:

Integra hardware, software y comunicación IoT en una plataforma Raspberry Pi mediante el desarrollo de un sistema embebido de monitoreo y automatización, utilizando Python para el control de GPIO, sensores, actuadores, MQTT, MongoDB Atlas y un dashboard web para resolver un problema de supervisión y control de un invernadero inteligente.

1.3. Habilidad(es) blandas a formar

El proyecto le permitirá desarrollar las siguientes habilidades:

- Trabajo en equipo.
- Organización y distribución de responsabilidades.
- Resolución de problemas técnicos.
- Comunicación de resultados.
- Integración de componentes desarrollados por distintos miembros del equipo.
- Documentación técnica de sistemas embebidos.
- Defensa técnica individual y grupal.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Diseñar e implementar un sistema IoT de automatización para un invernadero dividido en dos áreas de cultivo y un centro de control.	Integrando sensores ambientales, sensores de suelo, sensor de gas, LDR, bomba de agua, ventilador, LEDs, buzzer, LCD, botones físicos, MQTT, MongoDB y dashboard web.	Mediante Raspberry Pi, Python, GPIO, MQTT, MongoDB Atlas y una interfaz web de monitoreo y control.	Para relacionar sistemas embebidos, automatización agrícola, comunicación IoT y visualización de datos dentro de una solución funcional.	Durante el período establecido para la Fase 1 del proyecto, equivalente a 2 semanas.

3. Resumen Ejecutivo

La primera fase del proyecto consiste en el diseño e implementación de un invernadero inteligente automatizado utilizando una Raspberry Pi 3 o 4 como unidad principal de control. El sistema deberá integrar sensores ambientales, sensores de humedad de suelo, sensor de gas, sensor de luz, actuadores físicos, una pantalla LCD, botones de control local, comunicación MQTT, almacenamiento en MongoDB Atlas y un dashboard web para monitoreo y control remoto.

El invernadero deberá construirse como una maqueta física funcional, dividida en dos áreas de cultivo y un centro de control. Cada área deberá contar con sensores y actuadores distribuidos de forma coherente, de manera que el sistema pueda monitorear condiciones reales del entorno y ejecutar respuestas automáticas.

Durante esta fase, toda la lógica de control se desarrollará en Python, utilizando GPIO para interactuar con sensores y actuadores. El sistema deberá ser capaz de leer datos, tomar decisiones automáticas mediante umbrales, activar riego real, controlar ventilación, iluminación, alarmas y mostrar información tanto en una pantalla LCD como en un dashboard web.

La finalidad de esta fase es que los estudiantes integren hardware, software, comunicación IoT, persistencia de datos y visualización web en una solución embebida funciona..

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

En un invernadero real es necesario monitorear constantemente las condiciones ambientales y del suelo para garantizar el crecimiento adecuado de las plantas. Variables como temperatura, humedad ambiental, humedad del suelo, iluminación y presencia de gases pueden afectar directamente el estado del cultivo.

Además, un sistema moderno de invernadero debe ser capaz de ejecutar acciones automáticas como riego, ventilación, iluminación artificial y activación de alarmas. También debe permitir supervisión local y remota, registrar datos históricos y ofrecer información clara para la toma de decisiones.

Para resolver esta necesidad, los estudiantes deberán construir un sistema IoT sobre Raspberry Pi que permita controlar un invernadero dividido en dos áreas de cultivo y un centro de control físico.

4.2 Alcance del proyecto

Estructura física obligatoria de la maqueta

La maqueta deberá representar físicamente un invernadero funcional. No se aceptará únicamente una protoboard con sensores conectados.

La maqueta deberá estar dividida en tres zonas visibles: Cada área de cultivo deberá estar claramente identificada y deberá tener elementos físicos que representen el funcionamiento del sistema: plantas reales o simuladas, suelo o recipiente de cultivo, mangueras o tubería de riego, LEDs de iluminación y estructura visible del invernadero.

Zona	Descripción
Área de cultivo 1	Primera sección del invernadero, con planta, sensor de humedad de suelo y sistema de riego.
Área de cultivo 2	Segunda sección del invernadero, con planta, sensor de humedad de suelo y sistema de riego.
Centro de control	Zona donde se ubican la pantalla LCD, botones físicos, LEDs de estado, buzzer y Raspberry Pi.

Cada área de cultivo deberá estar claramente identificada y deberá tener elementos físicos que representen el funcionamiento del sistema: plantas reales o simuladas, suelo o recipiente de cultivo, mangueras o tubería de riego, LEDs de iluminación y estructura visible del invernadero.

Alcance obligatorio:

La Fase 1 deberá cumplir obligatoriamente con los siguientes elementos:

1. Construcción de una maqueta física de invernadero con dos áreas de cultivo y un centro de control.
2. Uso de Raspberry Pi 3 o 4 como unidad principal.
3. Implementación de la lógica de control en Python.
4. Integración de sensores ambientales, sensores de suelo, sensor de luz y sensor de gas.
5. Implementación de riego real mediante bomba de agua o sistema equivalente funcional.
6. Control automático de ventilación, iluminación, riego y alarmas.
7. Uso obligatorio de MQTT (MQTTX) para comunicación de datos y comandos.
8. Uso obligatorio de MongoDB Atlas para persistencia.
9. Desarrollo de dashboard web para monitoreo y control remoto.
10. Implementación de pantalla LCD en el centro de control.
11. Implementación de botones físicos para control local.
12. Registro de lecturas, eventos, estados y comandos.
13. Video demostrativo y documentación técnica.

Subsistemas obligatorios

Subsistema	Componentes	Función
Monitoreo ambiental	DHT11 o DHT22	Medir temperatura y humedad ambiental general del invernadero.
Monitoreo de suelo	2 sensores de humedad de suelo	Determinar si el suelo de cada área está seco, normal o saturado.
Iluminación inteligente	LDR + LEDs blancos	Encender iluminación artificial cuando haya poca luz.

Ventilación	Ventilador DC o extractor pequeño	Activarse cuando exista temperatura alta o gas detectado.
Riego funcional	Bomba de agua + mangueras + recipiente	Activar riego real en el área que tenga suelo seco.
Detección de gas	MQ-2 o MQ-135	Detectar presencia de gas, humo o mala calidad del aire.
Panel local	LCD 16x2 o 20x4 + botones	Mostrar lecturas, estados y permitir control manual.
Alarma	Buzzer + LED rojo	Alertar condiciones críticas.
Estado visual	LEDs verde, amarillo y rojo	Representar estado global del invernadero.
Comunicación IoT	MQTT (MQTTX)	Enviar lecturas y recibir comandos.
Persistencia	MongoDB Atlas	Guardar historial de lecturas, eventos y comandos.
Dashboard	Web	Visualizar datos y controlar actuadores remotamente.

Detalle de subsistemas obligatorios

1. Sistema de monitoreo ambiental

Componentes obligatorios

- Sensor DHT11 o DHT22.
- Raspberry Pi.
- Python para lectura del sensor.

Funcionalidad requerida

El sistema deberá medir periódicamente:

- temperatura ambiental;
- humedad ambiental.

Cada lectura deberá:

- publicarse por MQTT;
- almacenarse en MongoDB Atlas;
- mostrarse en el dashboard;
- mostrarse en la pantalla LCD de forma rotativa;
- utilizarse para determinar el estado global del invernadero.

Umbrales mínimos

Se deberá definir al menos:

Variable	Estado normal	Estado de advertencia
Temperatura	Rango definido por el equipo	Temperatura alta
Humedad ambiental	Rango definido por el equipo	Humedad baja o alta

Cuando la temperatura supere el umbral definido:

- se deberá activar ventilación;
- se deberá registrar evento en MongoDB;
- se deberá cambiar el estado global a ADVERTENCIA;
- se deberá mostrar alerta en dashboard y LCD.

2. Sistema de monitoreo de suelo por área

Componentes obligatorios

- 2 sensores de humedad de suelo.
- Uno para el Área de cultivo 1.
- Uno para el Área de cultivo 2.

Funcionalidad requerida

Cada área de cultivo deberá monitorearse de forma independiente.

El sistema deberá clasificar el suelo de cada área como:

- SECO
- NORMAL
- SATURADO

Cada lectura deberá:

- publicarse mediante MQTT;
- almacenarse en MongoDB;
- mostrarse en dashboard;
- mostrarse en LCD;
- utilizarse para activar riego automático.

Reglas mínimas

Si el suelo del Área 1 está seco:

- se activa el riego del Área 1;
- se registra evento;
- se actualiza el dashboard;
- se muestra mensaje en LCD.

Si el suelo del Área 2 está seco:

- se activa el riego del Área 2;
- se registra evento;
- se actualiza el dashboard;
- se muestra mensaje en LCD.

Si el suelo está saturado:

- no se debe activar riego;
- se debe mostrar advertencia;
- se debe registrar evento.

3. Sistema de riego funcional

Componentes obligatorios

- Bomba de agua funcional.
- Recipiente de agua.
- Mangueras o tubería.
- Relé, transistor o módulo de control según el montaje.
- Fuente externa si la bomba lo requiere.

Funcionalidad requerida

El riego debe ser real y funcional.

El sistema deberá permitir:

1. Riego automático por humedad baja.
2. Riego manual desde botones físicos.
3. Riego remoto desde dashboard.
4. Registro de cada activación de riego.
5. Protección contra activaciones continuas.

Reglas obligatorias

- La bomba no debe permanecer encendida indefinidamente.
- Cada activación deberá tener una duración controlada.
- El sistema debe impedir riego si el suelo está saturado.
- Debe existir una pausa mínima entre ciclos de riego.
- El dashboard debe mostrar si el riego está activo o inactivo.
- La LCD debe indicar cuando el sistema esté regando.

Estados del riego

- RIEGO_OFF
- RIEGO_AREA_1
- RIEGO_AREA_2
- RIEGO_MANUAL
- BLOQUEADO_POR_SATURACION

4. Sistema de iluminación inteligente

Componentes obligatorios

- Sensor LDR.
- Mínimo 2 LEDs+ blancos o tira LED.
- Un LED o grupo de LEDs por cada área de cultivo.

Funcionalidad requerida

El sistema deberá medir el nivel de luz ambiental y determinar si el invernadero necesita iluminación artificial.

Cuando el nivel de luz sea bajo:

- se deberán encender las luces;
- se deberá registrar el evento;
- se deberá actualizar el dashboard;
- se deberá mostrar el estado en la LCD.

Cuando el nivel de luz sea suficiente:

- se deberán apagar las luces automáticamente;
- se deberá actualizar el estado del sistema.

Modos de funcionamiento

- AUTOMATICO
- MANUAL

En modo automático, Python decide según el sensor LDR.

En modo manual, el usuario puede encender o apagar las luces desde botones físicos o dashboard.

5. Sistema de ventilación

Componentes obligatorios

- Ventilador DC, extractor pequeño.
- Transistor, relé o módulo de control según el componente usado.

Funcionalidad requerida

El sistema de ventilación deberá activarse cuando:

- la temperatura supere el umbral definido;
- el sensor de gas detecte condición peligrosa;
- el usuario lo active desde el dashboard;
- el usuario lo active desde el panel físico.

Estados

- VENTILACION_ON
- VENTILACION_OFF
- VENTILACION_MANUAL
- VENTILACION_EMERGENCIA

En caso de emergencia por gas, la ventilación debe activarse automáticamente aunque el sistema esté en modo manual.

6. Sistema de detección de gas o humo

Componentes obligatorios

- Sensor MQ-2 o MQ-135.
- Buzzer.
- LED rojo.
- Ventilador o extractor.

Funcionalidad requerida

El sensor deberá monitorear continuamente el nivel de gas, humo o calidad del aire.

Cuando el valor supere el umbral definido:

- se activa el buzzer;
- se enciende el LED rojo;
- se activa ventilación;
- se cambia el estado global a EMERGENCIA;
- se registra el evento en MongoDB;
- se muestra alerta en dashboard;
- se muestra mensaje fijo en LCD.

El sistema no podrá regresar automáticamente a NORMAL mientras el valor del sensor siga por encima del umbral.

Estados posibles

- GAS_NORMAL
- GAS_ADVERTENCIA
- GAS_EMERGENCIA

7. Centro de control físico

Componentes obligatorios

- Pantalla LCD 16x2 o 20x4.
- Mínimo 4 botones físicos.
- LEDs de estado verde, amarillo y rojo.
- Buzzer.
- Raspberry Pi.

Funcionalidad de la LCD

La pantalla LCD deberá mostrar de forma rotativa:

- temperatura;
- humedad ambiental;
- humedad del suelo del Área 1;
- humedad del suelo del Área 2;
- nivel de luz;
- nivel de gas;
- estado de riego;
- estado de ventilación;
- estado de iluminación;
- estado global del invernadero.

En caso de emergencia por gas o condición crítica, la pantalla deberá mostrar una alerta fija hasta que la condición desaparezca o sea atendida.

Botones físicos obligatorios

Botón	Función
Botón 1	Cambiar entre modo automático y modo manual.
Botón 2	Activar riego manual.
Botón 3	Encender/apagar luces manualmente.

Botón 4	Silenciar buzzer o restablecer alerta.
---------	--

Opcionalmente, se puede agregar un quinto botón para activar ventilación manual.

8. Sistema de estado global

El sistema deberá manejar un estado global que represente la condición general del invernadero.

Estados obligatorios

- NORMAL
- ADVERTENCIA
- RIEGO_ACTIVO
- MODO_MANUAL
- EMERGENCIA

Reglas mínimas

Estado	Condición
NORMAL	Todos los sensores dentro de rangos seguros.
ADVERTENCIA	Temperatura alta, poca luz o suelo seco.
RIEGO_ACTIVO	La bomba está funcionando.
MODO_MANUAL	El usuario controla actuadores manualmente.
EMERGENCIA	Gas/humo detectado por encima del umbral.

Indicadores físicos

LED	Estado
Verde	NORMAL
Amarillo	ADVERTENCIA o RIEGO_ACTIVADO
Rojo	EMERGENCIA

Comunicación MQTT obligatoria

El sistema deberá usar MQTT para comunicar lecturas, estados y comandos. No se aceptará sustituir MQTT por consultas directas entre componentes. (MQTTX)

Topics mínimos

- invernadero/sensores/temperatura
- invernadero/sensores/humedad_ambiente
- invernadero/sensores/humedad_suelo_area1
- invernadero/sensores/humedad_suelo_area2
- invernadero/sensores/luz
- invernadero/sensores/gas
- invernadero/actuadores/riego
- invernadero/actuadores/riego_area1
- invernadero/actuadores/riego_area2
- invernadero/actuadores/ventilador
- invernadero/actuadores/luces
- invernadero/actuadores/alarma
- invernadero/estado/global
- invernadero/control/remoto
- invernadero/control/manual

Comportamiento requerido

- Python deberá publicar lecturas de sensores.
- El dashboard deberá recibir información en tiempo real o mediante backend.
- El dashboard deberá enviar comandos de control remoto.
- La Raspberry Pi deberá recibir y ejecutar comandos.
- Los eventos relevantes deberán almacenarse en MongoDB.
- Todo comando remoto deberá quedar registrado.

Persistencia en MongoDB Atlas

Colecciones mínimas

Colección	Contenido mínimo
sensor_readings	Temperatura, humedad ambiental, humedad de suelo, luz y gas con timestamp.
events	Alertas, emergencias, activaciones de riego, ventilación, luces y cambios de estado.
commands	Comandos enviados desde dashboard o botones físicos.
system_status	Estado global actual del invernadero.
actuator_logs	Historial de activación de bomba, ventilador, luces, buzzer y LEDs.

Cada documento debe incluir como mínimo:

- fecha y hora;
- tipo de dato;
- valor;
- origen;
- estado relacionado.

Dashboard web obligatorio

El dashboard deberá incluir como mínimo:

Panel principal

- Estado global del invernadero.
- Última lectura de temperatura.
- Última lectura de humedad ambiental.
- Humedad del suelo del Área 1.

- Humedad del suelo del Área 2.
- Nivel de luz.
- Nivel de gas.
- Estado de riego.
- Estado de ventilación.
- Estado de luces.
- Estado de alarma.

Gráficas

Debe incluir gráficas históricas de:

- temperatura;
- humedad ambiental;
- humedad de suelo Área 1;
- humedad de suelo Área 2;
- nivel de luz;
- nivel de gas.

Controles remotos

Debe permitir:

- activar/desactivar riego;
- seleccionar área de riego;
- encender/apagar luces;
- activar/desactivar ventilación;
- silenciar alarma;
- cambiar modo automático/manual;
- restablecer estado cuando sea posible.

Historial

Debe mostrar:

- últimos eventos;
- últimos comandos;
- últimas alertas;
- últimas activaciones de actuadores.

Sección de análisis ARM64

El dashboard deberá incluir una sección específica llamada **Análisis ARM64**.

Esta sección deberá mostrar los resultados generados por los módulos ARM64 individuales de cada integrante.

Debe mostrar como mínimo:

Módulo	Resultado esperado
Media aritmética ponderada	Valor calculado sobre una variable seleccionada del archivo <code>lecturas.csv</code> .
Varianza y desviación estándar	Medidas de dispersión calculadas sobre 30 datos reales.
Detección de anomalías	Cantidad de anomalías detectadas y nivel de riesgo.
Predicción lineal simple	Valor inicial, valor final, diferencia total, promedio de cambio y predicción.
Tendencia acumulada avanzada	Incrementos, decrementos, rachas máximas y tendencia final.

9.Procesamiento de datos en ARM64

Debido a que el sistema del invernadero estará basado en Raspberry Pi, cada grupo deberá incorporar procesamiento desarrollado en lenguaje ensamblador ARM64/AArch64 para analizar datos ambientales directamente sobre la arquitectura utilizada por el dispositivo.

El procesamiento en ARM64/AArch64 se realizará mediante archivos fuente `.s`, los cuales deberán contener funciones externas o rutinas individuales encargadas de operar sobre información real generada por los sensores del invernadero. Estas rutinas permitirán ejecutar operaciones estadísticas y matemáticas utilizando programación de bajo nivel sobre la misma plataforma donde opera el sistema IoT.

Cada grupo estará conformado por 5 integrantes. Cada integrante deberá desarrollar, compilar, ejecutar, depurar y defender una rutina ARM64/AArch64 individual relacionada con el procesamiento de datos del invernadero. No se aceptará que un único integrante implemente todas las rutinas ARM64/AArch64 del grupo.

Todos los cálculos deberán realizarse únicamente sobre un conjunto de 30 datos reales tomados desde el archivo `lecturas.csv`.

9.1 Flujo obligatorio de integración

El sistema deberá seguir el siguiente flujo:

Sensores reales -> Python -> lecturas.csv -> Módulos ARM64 -> resultados_arm64/ -> Python -> MongoDB -> Dashboard

Python tendrá permitido:

- leer sensores;
- controlar actuadores;
- publicar datos por MQTT;
- almacenar datos en MongoDB;
- generar el archivo `lecturas.csv`;
- ejecutar los programas ARM64;
- leer los archivos de salida generados por ARM64;
- publicar los resultados en MongoDB y dashboard.

Python no podrá realizar los cálculos solicitados a las rutinas de ARM64.

9.2 Archivo de entrada obligatorio

Python deberá generar un archivo llamado `lecturas.csv` con lecturas reales tomadas del sistema.

El archivo deberá contener exactamente 30 registros reales.

Formato obligatorio:

ID,TEMP,HUM_AIRE,HUM_SUELO_1,HUM_SUELO_2,LUZ,GAS,RIEGO_1,RIEGO_2

1,28,70,45,48,320,120,0,0

2,29,68,42,47,300,130,0,0

3,31,65,38,43,250,145,1,0

4,33,60,35,40,220,180,1,0

...

30,30,66,41,44,260,150,0,0

\$

Todos los módulos ARM64 deberán leer este archivo como entrada.

9.3 Restricciones técnicas obligatorias para ARM64

Cada rutina ARM64 deberá cumplir con lo siguiente:

1. Estar desarrollado en ensamblador ARM64/AArch64.
2. Compilarse mediante `as`, `ld` o `gcc`.
3. Tener su propio archivo `.s`.
4. Tener una salida propia en archivo `.txt`.
5. Leer datos desde `lecturas.csv`.
6. Procesar únicamente los 30 datos del archivo.
7. Usar registros, memoria, ciclos y saltos condicionales.
8. Implementar conversión ASCII a entero.
9. Implementar conversión entero a ASCII para escribir resultados.
10. Usar al menos una subrutina propia.
11. Incluir evidencia de depuración con GDB.
12. Ser defendido individualmente por el integrante responsable.

9.4 Distribución de rutinas ARM64 por integrante

Cada integrante deberá implementar una rutina estadístico distinto.

Todas las rutinas ARM64 deberán utilizar una biblioteca común de utilidades (`utils.s`) encargada de:

- lectura del archivo `lecturas.csv`;
- recorrido de registros;
- separación de columnas;
- conversión ASCII a entero;
- almacenamiento temporal de datos;
- escritura de resultados en archivos `.txt`;
- manejo de buffers y memoria.

La biblioteca `utils.s` deberá ser utilizada por todos los módulos ARM64.

Todos los módulos deberán trabajar únicamente sobre una variable X formada por un conjunto de 30 datos reales.

10. Descripción de rutinas ARM64 obligatorios

Todas las rutinas deberán trabajar únicamente sobre una variable:

X = conjunto de 30 datos

Los datos podrán provenir de:

- temperatura;
- humedad;
- luz;
- gas;
- humedad de suelo.

10.1 Rutina 1: Media aritmética ponderada

Responsable: Integrante 1

Archivo: `modulo_1_media.s`

Salida: `resultado_media.txt`

Esta rutina deberá calcular una media aritmética ponderada utilizando pesos crecientes asociados a cada lectura.

La rutina deberá calcular:

- suma total de datos;
- suma ponderada;
- suma de pesos;
- media ponderada.

Fórmula obligatoria:

$$\text{MEDIA_PONDERADA} = \Sigma(X_i * W_i) / \Sigma W_i$$

Donde:

W_i = peso asociado a cada lectura

Los pesos deberán incrementarse progresivamente desde 1 hasta 30.

Formato de salida:

- `MODULE=WEIGHTED_MEAN`
- `TOTAL_VALUES=30`
- `SUM_X=920`
- `WEIGHT_SUM=465`
- `WEIGHTED_MEAN=31`

10.2 Rutina 2: Varianza y desviación estándar

Responsable: Integrante 2

Archivo: `modulo_2_varianza.s`

Salida: `resultado_varianza.txt`

Esta rutina deberá calcular medidas de dispersión estadística.

Esta rutina deberá calcular:

- media;
- varianza;
- desviación estándar.

Fórmulas obligatorias:

- $MEDIA = \Sigma X / N$
- $VAR = \Sigma (X - MEDIA)^2 / N$
- $DESV = \text{sqrt}(VAR)$

Donde:

$N = 30$

Formato de salida:

- `MODULE=VARIANCE`
- `TOTAL_VALUES=30`
- `MEAN=31`
- `VARIANCE=18`
- `STD_DEV=4`

10.3 Rutina 3: Detección estadística de anomalías

Responsable: Integrante 3

Archivo: `modulo_3_anomalias.s`

Salida: `resultado_anomalias.txt`

Esta rutina deberá detectar anomalías utilizando z-score.

Esta rutina deberá calcular:

- media;
- desviación estándar;
- z-score para cada dato;
- cantidad total de anomalías.

Fórmula obligatoria:

$$Z = (X - MEDIA) / DESVIACION_ESTANDAR$$

Regla obligatoria:

$$|Z| \geq 2 = \text{ANOMALIA}$$

Formato de salida:

- `MODULE=ANOMALY_DETECTION`
- `TOTAL_VALUES=30`
- `MEAN=29`
- `STD_DEV=3`
- `ANOMALIES=4`
- `SYSTEM_RISK=HIGH`

Clasificación:

- 0 anomalías = NORMAL
- 1-3 anomalías = MEDIUM
- 4 o más anomalías = HIGH

10.4 Rutina 4: Predicción lineal simple

Responsable: Integrante 4

Archivo: `modulo_4_prediccion.s`

Salida: `resultado_prediccion.txt`

Esta rutina deberá realizar una predicción simple utilizando tendencia promedio.

Esta rutina deberá calcular:

- valor inicial;
- valor final;
- diferencia total;
- promedio de cambio;
- predicción siguiente.

Fórmulas obligatorias:

- $DIF = X_{FINAL} - X_{INICIAL}$
- $PROMEDIO_CAMBIO = DIF / (N - 1)$
- $PREDICCION = X_{FINAL} + PROMEDIO_CAMBIO$

Donde:

$N = 30$

Formato de salida:

- `MODULE=PREDICTION`
- `INITIAL_VALUE=28`
- `FINAL_VALUE=34`
- `TOTAL_DIFF=6`
- `AVG_CHANGE=0.20`
- `NEXT_VALUE=34.20`

10.5 Rutina 5: Tendencia acumulada avanzada

Responsable: Integrante 5

Archivo: `modulo_5_tendencia.s`

Salida: `resultado_tendencia.txt`

Esta rutina deberá analizar la tendencia general de los 30 datos considerando estabilidad y cambios consecutivos.

Esta rutina deberá calcular:

- cantidad de incrementos consecutivos;
- cantidad de decrementos consecutivos;
- longitud máxima de crecimiento continuo;
- longitud máxima de decremento continuo;
- diferencia acumulada;
- tendencia final.

Fórmulas obligatorias:

- $DIF_i = X_i - X_{(i-1)}$
- $DIF_ACUM = \sum DIF_i$

Clasificación:

- $DIF_ACUM > 0 = UP$
- $DIF_ACUM < 0 = DOWN$
- $DIF_ACUM = 0 = STABLE$

Formato de salida:

- `MODULE=ADVANCED_TREND`
- `TOTAL_VALUES=30`
- `INCREMENTS=18`
- `DECREMENTS=10`
- `MAX_UP_STREAK=5`
- `MAX_DOWN_STREAK=3`
- `ACCUM_DIFF=7`
- `TREND=UP`

11. Integración de resultados ARM64

Python deberá ejecutar las cinco rutinas ARM64 y leer sus archivos de salida.

Todos los módulos deberán enlazarse con la biblioteca común:

`as utils.s -o utils.o`

Los resultados deberán almacenarse en una colección llamada:

`arm64_results`

12. Evidencia de autoría individual

Cada integrante deberá presentar evidencia individual de su rutina ARM64.

Cada integrante deberá entregar:

1. Archivo `.s` propio.
2. Explicación escrita del algoritmo implementado.
3. Explicación de registros utilizados.
4. Explicación de ciclos, saltos y subrutinas.
5. Explicación del formato de entrada y salida.
6. Commit individual asociado a su módulo.
7. Defensa oral individual.

13. Restricciones del apartado ARM64

Para que el apartado ARM64 sea válido, deberán cumplirse las siguientes restricciones:

1. No se permite calcular los resultados ARM64 en Python.
2. Todas las rutinas deberán trabajar con exactamente 30 datos.
3. Cada rutina debe leer datos reales generados por el invernadero.
4. Todas las rutinas deberán utilizar `utils.s`.
5. Cada rutina debe generar un archivo de salida propio.
6. Cada rutina debe compilar y ejecutarse de forma independiente.
7. Cada rutina debe ser defendido por su autor.
8. Se deberá demostrar al menos una sesión de depuración con GDB por integrante.
9. Los resultados deberán integrarse al dashboard y MongoDB.

Alcance opcional

Como funcionalidades opcionales se podrán agregar:

- sensor de nivel de agua para el depósito;
- autenticación de usuarios en el dashboard;
- notificaciones por correo o Telegram;
- cámara para monitoreo visual del invernadero;
- exportación de reportes;
- control independiente de riego por área con válvulas;
- gráficas comparativas entre Área 1 y Área 2;
- modo nocturno automático;
- registro de consumo de agua.

4.3 Recursos y herramientas a utilizar

Tipo	Categoría	Descripción
------	-----------	-------------

Obligatorio	Plataforma	Raspberry Pi 3 o 4
Obligatorio	Lenguaje	Python 3
Obligatorio	Hardware	Sensor DHT11 o DHT22
Obligatorio	Hardware	2 sensores de humedad de suelo
Obligatorio	Hardware	Sensor LDR
Obligatorio	Hardware	Sensor MQ-2 o MQ-135
Obligatorio	Hardware	Bomba de agua funcional
Obligatorio	Hardware	Ventilador DC o extractor
Obligatorio	Hardware	LEDs de iluminación
Obligatorio	Hardware	LED verde, amarillo y rojo
Obligatorio	Hardware	Buzzer
Obligatorio	Hardware	LCD 16x2 o 20x4
Obligatorio	Hardware	Mínimo 4 botones físicos

Obligatorio	Comunicación	MQTT (MQTTX)
Obligatorio	Base de datos	MongoDB Atlas
Obligatorio	Software	Dashboard web
Recomendado	Software	Flask, FastAPI o Node.js
Recomendado	Software	React, Vue o HTML/JS
Recomendado	Herramientas	GitHub, VS Code, Draw.io
Recomendado	Hardware	Fuente externa para bomba o ventilador
Recomendado	Hardware	Relé, transistor o driver para actuadores

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Maqueta física	Invernadero dividido en dos áreas de cultivo y centro de control.
Código fuente	Scripts Python para sensores, actuadores, MQTT y MongoDB.

Dashboard web	Interfaz funcional con gráficas, estados y controles remotos.
Base de datos	Evidencia de colecciones MongoDB con datos reales.
Comunicación MQTT	Evidencia de topics funcionando.
Panel local	LCD y botones físicos funcionando.
Sistema de riego	Bomba de agua funcional integrada a la maqueta.
Diagramas	Diagrama de arquitectura, conexiones físicas y flujo de datos.
Código ARM64	Cinco archivos <code>.s</code> , uno por integrante.
Biblioteca común	Archivo <code>utils.s</code> utilizado por todos los módulos.
Makefile	Debe compilar todos los módulos ARM64.
Archivo de entrada	<code>lecturas.csv</code> generado desde datos reales del invernadero.
Integración Python	Script que ejecute los módulos ARM64 y lea sus resultados.
MongoDB	Colección <code>arm64_results</code> con resultados de los cinco módulos.
Documentación individual	Explicación del algoritmo, registros, memoria y flujo.
Video demostrativo	Video que muestre sensores, actuadores, riego, dashboard, MQTT y MongoDB.
Documentación técnica	Informe con explicación de subsistemas, topics, colecciones y decisiones de diseño.

5. Material de apoyo

Categoría	Recomendado	Descripción
Documentación oficial	Documentación de Raspberry Pi GPIO	Uso de pines GPIO y conexión de componentes.
Documentación oficial	Documentación de Python	Uso del lenguaje para control de hardware.
Documentación oficial	Documentación de MQTT / Mosquitto	Comunicación por topics entre Raspberry Pi y dashboard.
Documentación oficial	Documentación de MongoDB Atlas	Persistencia de lecturas, eventos y comandos.
Manual / Bibliografía	Material de sensores DHT, MQ, LDR y humedad de suelo	Referencia para conexión y lectura de sensores.
Ejemplo	Guías proporcionadas en el curso	Ejemplos de Raspberry Pi, Python, MQTT y dashboard.

6. Metodología

A continuación se presenta una metodología sugerida para elaborar el proyecto:

1. Análisis y diseño del sistema

Definir la estructura física del invernadero, las dos áreas de cultivo, el centro de control, los sensores, actuadores, topics MQTT, colecciones MongoDB, arquitectura general del sistema y distribución de responsabilidades por integrante.

En esta etapa también se deberá asignar el módulo ARM64 que desarrollará cada integrante del grupo.

2. Construcción de la maqueta física

Diseñar y ensamblar la estructura del invernadero, ubicando de forma visible las dos áreas de cultivo, el sistema de riego, luces, ventilación, sensores y centro de control.

La maqueta deberá permitir demostrar físicamente el funcionamiento del sistema, especialmente el riego real, la iluminación, la ventilación, la alarma y la interacción con la pantalla LCD y botones físicos.

3. Pruebas individuales de hardware

Verificar cada sensor y actuador por separado antes de integrarlo al sistema completo. Se deberán probar sensores de suelo, DHT11/DHT22, LDR, MQ-2/MQ-135, bomba de agua, ventilador, LEDs, buzzer, LCD y botones.

Cada prueba deberá validar que el componente pueda ser leído o controlado correctamente desde Python utilizando GPIO.

4. Implementación del núcleo en Python

Programar la lectura de sensores, control de actuadores, reglas de estado global, modo automático/manual, publicación MQTT y recepción de comandos.

Python será responsable de controlar el hardware físico del invernadero, generar lecturas reales y coordinar la integración con los demás componentes del sistema.

5. Configuración de MQTT

Definir los topics, probar publicación de lecturas y recepción de comandos. Se deberá evidenciar comunicación funcional entre Raspberry Pi, dashboard y demás componentes del sistema.

MQTT deberá utilizarse para enviar lecturas de sensores, estados del sistema, comandos remotos y eventos relevantes.

6. Configuración de MongoDB Atlas

Crear las colecciones necesarias y registrar lecturas, eventos, comandos, estados, activaciones de actuadores y resultados generados por los módulos ARM64.

La base de datos deberá permitir consultar el historial del sistema y evidenciar que las lecturas y resultados provienen de ejecuciones reales.

7. Desarrollo del dashboard web

Implementar visualización de datos actuales, gráficas históricas, estado global, controles remotos e historial de eventos.

El dashboard deberá incluir una sección específica para visualizar los resultados generados por los módulos ARM64, tales como media ponderada, varianza, anomalías, predicción y tendencia.

8. Integración del centro de control físico

Integrar LCD, botones físicos, LEDs de estado y buzzer con la lógica general del sistema.

La LCD deberá mostrar datos rotativos del invernadero y alertas críticas. Los botones deberán permitir acciones locales como cambio de modo, riego manual, control de luces y silenciamiento de alarma.

9. Generación del archivo de datos para ARM64

Python deberá generar el archivo `lecturas.csv` utilizando exactamente 30 lecturas reales obtenidas desde los sensores del invernadero.

El archivo deberá respetar el formato definido en el enunciado y será la entrada obligatoria para todos los módulos ARM64. No se permitirá utilizar datos inventados manualmente como fuente principal de evaluación.

10. Desarrollo de biblioteca común `utils.s`

El grupo deberá desarrollar una biblioteca común en ARM64 llamada `utils.s`, utilizada por todos los módulos individuales.

Esta biblioteca deberá encargarse de tareas comunes como:

- lectura del archivo `lecturas.csv`;
- recorrido de registros;
- separación de columnas;
- conversión ASCII a entero;
- almacenamiento temporal de datos;
- escritura de resultados;
- manejo básico de buffers y memoria.

11. Desarrollo de módulos ARM64 individuales

Cada integrante deberá desarrollar un módulo ARM64 propio, con archivo `.s` independiente, orientado a una operación matemática o estadística distinta.

Los módulos definidos son:

1. Media aritmética ponderada.
2. Varianza y desviación estándar.
3. Detección estadística de anomalías.
4. Predicción lineal simple.
5. Tendencia acumulada avanzada.

Cada módulo deberá compilar, ejecutarse, generar su archivo de salida y ser defendido individualmente por su autor.

12. Integración de resultados ARM64

Python deberá ejecutar los cinco módulos ARM64, leer sus archivos de salida y almacenar los resultados en MongoDB dentro de la colección `arm64_results`.

Posteriormente, el dashboard deberá mostrar los resultados del análisis ARM64 en una sección específica.

13. Pruebas de integración final

Validar el flujo completo del sistema:

Sensores -> Python -> MQTT -> MongoDB -> Dashboard -> Comandos -> Raspberry Pi
-> Actuadores -> lecturas.csv -> ARM64 -> Resultados -> MongoDB -> Dashboard

También se deberá validar que los módulos ARM64 no realicen cálculos simulados y que Python no calcule los resultados que corresponden al apartado ARM64.

14. Depuración, documentación y presentación

Cada integrante deberá presentar evidencia de depuración con GDB de su módulo ARM64. Además, el grupo deberá preparar el informe técnico, diagramas, evidencias, video demostrativo y defensa del proyecto completo.

La documentación deberá explicar tanto la arquitectura IoT como el flujo de procesamiento ARM64.

Recomendaciones

- Probar cada módulo por separado antes de integrarlo.
- Usar fuente externa para bomba y ventilador si consumen más corriente que la permitida por la Raspberry Pi.
- No alimentar la bomba directamente desde GPIO.
- Documentar claramente los topics MQTT.
- Mantener ordenado el cableado.
- Definir umbrales realistas para temperatura, humedad, luz y gas.
- Verificar que el riego sea funcional y no solo simulado.
- Generar `lecturas.csv` únicamente con datos reales del sistema.
- Desarrollar primero `utils.s` antes de los módulos individuales.
- Validar cada módulo ARM64 con GDB.
- Asegurar que cada integrante pueda explicar su propio código ARM64.
- Registrar evidencias desde el inicio del proyecto.
- No dejar todo a última hora.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación del proyecto	Día 1	Día 1
Diseño de arquitectura general y distribución de responsabilidades	Día 1	Día 2
Construcción de maqueta física y pruebas de sensores/actuadores	Día 2	Día 5
Implementación de lógica Python GPIO, estados y control automático/manual	Día 4	Día 8
Integración MQTT, MongoDB y dashboard web	Día 6	Día 10
Generación de datos reales y archivo <code>lecturas.csv</code>	Día 8	Día 10
Desarrollo de biblioteca común <code>utils.s</code> y módulos ARM64 individuales	Día 9	Día 12
Integración ARM64 con Python, MongoDB y dashboard	Día 11	Día 12
Pruebas finales, GDB, documentación y video demostrativo	Día 12	Día 12
Evaluación y defensa técnica	Día 13	Día 13

8. Rúbrica de Calificación

En esta sección se aborda todo lo relacionado a la calificación, lo que permite claridad para el estudiante y el tutor académico de los aspectos a evaluar y los mismos que conocen desde el inicio por lo que todo proyecto debe incluirlos desde el inicio.

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Tema	Descripción	Cumple
Maqueta física	Presenta un invernadero físico con dos áreas de cultivo y centro de control.	
Raspberry Pi funcional	El sistema corre sobre Raspberry Pi 3 o 4.	
Sensores obligatorios	Integra DHT11/DHT22, 2 sensores de suelo, LDR y MQ-2/MQ-135.	
Riego funcional	La bomba de agua funciona realmente y riega al menos un área.	
Actuadores	Ventilador, luces, buzzer y LEDs de estado funcionan correctamente.	
LCD y botones	La pantalla muestra datos del sistema y los botones ejecutan acciones locales.	
Python GPIO	El control físico de sensores y actuadores se realiza desde Python.	
MQTT	Se publican lecturas y se reciben comandos mediante MQTT (MQTTX).	
MongoDB Atlas	Se almacenan lecturas, eventos, comandos, estados y resultados ARM64.	

Dashboard	Muestra datos, gráficas, controles remotos, historial y sección de análisis ARM64.	
Archivo <code>lecturas.csv</code>	Python genera un archivo <code>lecturas.csv</code> con exactamente 30 lecturas reales del invernadero.	
Módulos ARM64 individuales	Cada integrante presenta un archivo <code>.s</code> propio con una operación matemática distinta.	
Biblioteca común <code>utils.s</code>	Los módulos ARM64 utilizan una biblioteca común para lectura, parsing, conversión y manejo de datos.	
Compilación y ejecución ARM64	Todos los módulos ARM64 compilan, se ejecutan y generan su archivo de salida correspondiente.	
Integración ARM64	Python ejecuta los módulos ARM64, lee sus resultados, los guarda en MongoDB y los muestra en el dashboard.	
Evidencia GDB	Cada integrante presenta evidencia de depuración con GDB de su módulo ARM64.	
Defensa individual	Cada integrante puede explicar su módulo ARM64, registros usados, ciclos, saltos, memoria y algoritmo.	
Evidencia general	Presenta video demostrativo, repositorio, documentación técnica y diagramas del sistema completo.	
El auxiliar es colaborador del repositorio	El auxiliar debe ser colaborador del repositorio, de lo contrario no tienen derecho a calificación. Nombre del Repositorio: ARQUI1V1S_G#	

	Github del auxiliar: PoncheDeFrutas	
--	-------------------------------------	--

8.2 Detalle de la Calificación

Descripción de Ponderación	Valor	Observación	Punteo
HABILIDADES	10		
Repositorio ordenado y commits individuales	3	El repositorio debe incluir código Python, dashboard, configuración MQTT, conexión MongoDB, carpeta ARM64 y commits verificables por integrante.	
Documentación técnica y diagramas	3	Debe incluir arquitectura general, conexión física, flujo MQTT, modelo MongoDB y flujo Python -> ARM64 -> MongoDB -> Dashboard.	
Defensa técnica individual y grupal	4	Cada integrante debe explicar su aporte, su módulo ARM64, el funcionamiento general del sistema y las decisiones técnicas implementadas.	
Sub-Total Habilidades	10		
CONOCIMIENTOS	90		
Maqueta física e integración del invernadero	8		
Construcción física del invernadero con dos áreas de cultivo y centro de control	3	La maqueta debe representar claramente las dos áreas de cultivo y el centro de control. No se aceptan únicamente componentes en protoboard.	
Distribución ordenada de sensores, actuadores, riego, iluminación y cableado	3	Los componentes deben estar integrados de forma visible, funcional, segura y coherente con la maqueta.	

Estabilidad física del montaje durante la evaluación	2	El sistema debe poder demostrarse sin desconexiones constantes, falsos contactos o fallos físicos repetitivos.	
Sensores y adquisición de datos reales	12		
Lectura de temperatura y humedad ambiental con DHT11/DHT22	2	Las lecturas deben obtenerse desde Python y mostrarse en el sistema.	
Lectura independiente de humedad de suelo en Área 1 y Área 2	3	Cada área debe tener su propio sensor y clasificación de suelo: seco, normal o saturado.	
Lectura de nivel de luz mediante LDR	2	La lectura debe utilizarse para controlar la iluminación automática.	
Lectura de gas o humo mediante MQ-2/MQ-135	2	El sensor debe activar advertencia o emergencia según el umbral definido.	
Publicación y almacenamiento de lecturas reales con timestamp	3	Las lecturas deben publicarse por MQTT, almacenarse en MongoDB y usarse para generar <code>lecturas.csv</code> .	
Actuadores y control físico del invernadero	15		
Sistema de riego real con bomba de agua funcional	4	El riego debe funcionar físicamente. No se acepta simulación únicamente con LED.	
Control de riego por área y protección contra riego continuo	3	El sistema debe activar riego según el área afectada, evitar riego indefinido e impedir riego si el suelo está saturado.	
Control de ventilación por temperatura alta o gas detectado	2	El ventilador o extractor debe activarse automáticamente ante condiciones definidas.	
Control de iluminación automática y manual	2	Las luces deben responder al sensor LDR y también permitir control manual.	
Alarma física con buzzer, LED rojo y LEDs de estado	2	Debe existir señalización visual y auditiva para condiciones críticas.	
Funcionamiento de botones físicos para acciones locales	2	Los botones deben permitir acciones mínimas: modo manual/automático, riego, luces y reset/silencio.	

Centro de control físico	6		
Pantalla LCD con información rotativa	3	La LCD debe mostrar temperatura, humedad, suelo por área, luz, gas, riego, ventilación, luces y estado global.	
Alertas críticas visibles en LCD	1	En emergencia, la pantalla debe mostrar una alerta fija o prioritaria.	
LEDs indicadores de estado global	1	Los LEDs deben representar correctamente NORMAL, ADVERTENCIA, RIEGO_ACTIVO o EMERGENCIA.	
Control local consistente con el estado del sistema	1	Las acciones desde botones deben reflejarse en Python, MongoDB y dashboard.	
Lógica de control en Python	10		
Lectura y control de GPIO desde Python	3	Python debe encargarse de leer sensores y controlar actuadores físicos.	
Implementación de modo automático y modo manual	2	El sistema debe diferenciar decisiones automáticas y acciones manuales del usuario.	
Manejo de estados globales del invernadero	2	Deben implementarse estados como NORMAL, ADVERTENCIA, RIEGO_ACTIVO, MODO_MANUAL y EMERGENCIA.	
Aplicación correcta de umbrales y reglas de decisión	2	Las respuestas del sistema deben depender de valores reales de sensores.	
Registro de eventos y comandos ejecutados	1	Cada evento importante debe quedar registrado en MongoDB.	
Comunicación IoT y persistencia de datos	12		
Implementación funcional de MQTT	4	Deben existir topics para sensores, actuadores, estado global, control remoto y control manual.	
Recepción y ejecución de comandos desde dashboard hacia Raspberry Pi	2	La Raspberry Pi debe recibir comandos remotos y ejecutarlos sobre los actuadores.	

Persistencia en MongoDB Atlas	3	Deben almacenarse lecturas, eventos, comandos, estados, activaciones de actuadores y resultados ARM64.	
Organización correcta de colecciones y documentos	2	Los documentos deben incluir timestamp, origen, valor, tipo de dato y estado relacionado.	
Evidencia de flujo IoT funcional	1	Debe comprobarse el flujo sensores -> Python -> MQTT -> MongoDB -> dashboard -> actuadores.	
Dashboard web funcional	7		
Panel principal con lecturas actuales y estado global	2	Debe mostrar temperatura, humedad ambiental, suelo por área, luz, gas, riego, ventilación, luces y alarma.	
Gráficas históricas de sensores	1	Debe mostrar gráficas de temperatura, humedad, suelo, luz y gas con datos reales almacenados.	
Controles remotos funcionales	2	Debe permitir controlar riego, luces, ventilación, alarma y modo automático/manual.	
Historial de eventos y comandos	1	Debe mostrar últimos eventos, comandos, alertas y activaciones de actuadores.	
Sección de resultados ARM64	1	Debe mostrar resultados principales de los cinco módulos ARM64.	
Módulos ARM64 individuales	25		
Biblioteca común <code>utils.s</code>	5	Debe permitir lectura de <code>lecturas.csv</code> , parsing, conversión ASCII-entero, manejo de datos y escritura básica de resultados.	
Módulo ARM64 Integrante 1: media aritmética ponderada	4	Debe compilar, ejecutarse, procesar datos reales, generar salida y ser defendido por su autor.	
Módulo ARM64 Integrante 2: varianza y desviación estándar	4	Debe compilar, ejecutarse, procesar datos reales, generar salida y ser defendido por su autor.	

Módulo ARM64 Integrante 3: detección estadística de anomalías	4	Debe compilar, ejecutarse, procesar datos reales, generar salida y ser defendido por su autor.	
Módulo ARM64 Integrante 4: predicción lineal simple	4	Debe compilar, ejecutarse, procesar datos reales, generar salida y ser defendido por su autor.	
Módulo ARM64 Integrante 5: tendencia acumulada avanzada	4	Debe compilar, ejecutarse, procesar datos reales, generar salida y ser defendido por su autor.	
Integración ARM64 con el sistema IoT	10		
Generación de <code>lecturas.csv</code> desde Python con 30 datos reales	2	El archivo debe generarse desde lecturas reales del invernadero.	
Ejecución automática de módulos ARM64 desde Python	2	Python debe ejecutar los binarios ARM64 desde el flujo del sistema.	
Lectura de archivos de salida generados por ARM64	2	Python debe leer los resultados <code>.txt</code> generados por cada módulo.	
Almacenamiento de resultados ARM64 en MongoDB	2	Debe existir colección <code>arm64_results</code> .	
Visualización de resultados ARM64 en dashboard	1	El dashboard debe mostrar los resultados principales del análisis ARM64.	
Evidencia de depuración con GDB por integrante	1	Cada integrante debe mostrar evidencia básica de depuración de su módulo ARM64.	
Funcionamiento global del sistema	5		
Integración completa de subsistemas	3	El sistema debe operar de forma coordinada entre sensores, Python, actuadores, MQTT, MongoDB, dashboard y ARM64.	
Estabilidad durante la evaluación	2	El sistema debe poder demostrarse de forma continua sin fallas generales que impidan su revisión.	

Sub-Total Conocimientos	90		
Total	100		

8.3 Comentarios Generales

Durante la evaluación, cualquier integrante podrá ser consultado sobre:

- funcionamiento de sensores y actuadores
- lógica implementada en Python
- conexión y uso de GPIO
- estructura de topics MQTT
- estructura de MongoDB Atlas
- dashboard y control remoto
- funcionamiento del sistema de riego
- funcionamiento de LCD y botones
- decisiones de diseño del proyecto.

No bastará con que el sistema funcione, el equipo deberá demostrar comprensión técnica de lo implementado.

Situación	Penalización
Presentar únicamente componentes conectados en protoboard, sin maqueta física de invernadero	Penalización grupal: 100% del proyecto. No se acepta como entrega válida.
No presentar maqueta física con dos áreas de cultivo y centro de control	Penalización grupal: no opta a calificación completa del proyecto.
No presentar repositorio de código (El Auxiliar no tiene acceso)	Penalización grupal: no se califica el proyecto.

No presentar documentación técnica	Penalización grupal: -15% sobre la nota final del proyecto.
Presentar documentación incompleta, desactualizada o que no corresponde al sistema implementado	Penalización grupal: -5% a -10% sobre la nota final del proyecto, según gravedad.
No presentar módulo ARM64 individual	El grupo obtiene 0 puntos en el módulo ARM64 correspondiente. Además, el integrante responsable recibe una penalización individual de hasta -50% sobre su nota final del proyecto.
El módulo ARM64 individual no compila	El grupo obtiene 0 puntos en el módulo ARM64 correspondiente. Además, el integrante responsable recibe una penalización individual de hasta -25% sobre su nota final del proyecto.
El módulo ARM64 individual compila, pero no ejecuta correctamente o no genera archivo de salida	El grupo obtiene 0 puntos en el módulo ARM64 correspondiente. Además, el integrante responsable recibe una penalización individual de hasta -10% sobre su nota final del proyecto.
El estudiante no puede explicar su código ARM64, registros, ciclos, saltos, memoria o fórmula implementada	Se asume falta de autoría. El grupo obtiene 0 puntos en el módulo ARM64 correspondiente y el integrante responsable recibe una penalización individual de hasta -70% sobre su nota final del proyecto.
El estudiante no puede defender cambios simples en su módulo ARM64 durante la evaluación	El integrante responsable recibe una penalización individual de hasta -50% sobre su nota final del proyecto.

Python realiza el cálculo que debía realizar ARM64	Penalización individual o grupal según corresponda: 0 puntos en el módulo ARM64 afectado, 0 puntos en integración ARM64 y hasta -40% sobre la nota final de los responsables.
Un integrante desarrolla el módulo ARM64 de otro integrante	Penalización individual: el integrante que no realizó su módulo obtiene 0 puntos en ARM64 y hasta -60% sobre su nota final individual. El integrante que realizó el módulo ajeno queda sujeto a penalización de hasta -50% sobre su nota final individual.
No poder explicar una parte general del sistema que fue asignada al integrante	Penalización individual: 0 puntos en el criterio asociado y hasta -5% sobre su nota final individual.
Participación parcial de un integrante	Penalización individual: recibe máximo 80% de la nota grupal.
Participación limitada de un integrante	Penalización individual: recibe máximo 60% de la nota grupal.
Participación mínima de un integrante	Penalización individual: recibe máximo 40% de la nota grupal.
Sin participación verificable	Penalización individual: recibe 0% de la nota grupal.

Durante la evaluación, cada integrante deberá demostrar dominio de su aporte individual. En el caso de ARM64, cada estudiante deberá defender su propio módulo, explicar el algoritmo implementado, justificar el uso de registros y memoria, y demostrar que el programa procesa datos reales generados por el invernadero.

El funcionamiento grupal del sistema no garantiza la nota individual completa si un integrante no puede demostrar autoría o comprensión de su parte asignada.